

基于几何锚定的视觉大模型与轻量模型协同的复杂金融文档还原方案

AFAC2026 挑战组赛题二·方法描述

队伍 noone ·杨康、赖坤豪

2026-08

1. 任务与核心判断

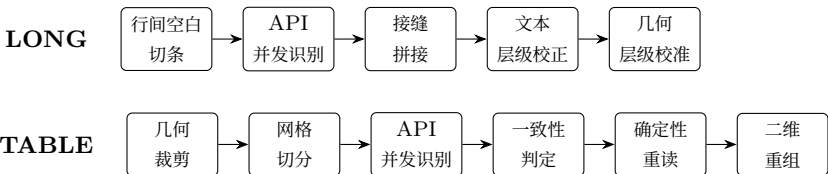
赛题要求把超长/超大金融文档截图还原为高保真 Markdown，评分由文本编辑距离、表格 TEDS、阅读流编辑距离三项等权构成。唯一可用的视觉语言模型是主办方提供的 FinixDoc-VL API，不可微调、不可替换。

方案设计的前提是先摸清这个黑盒的行为边界。正式开发前用标定实验测得四条硬约束：

特性	实测结论	对方案的约束
接口形态	只接受图片，无 prompt 等文本参数	结构控制只能在客户端
单次输出上限	约 10–12k 字符，与图像尺寸无关	调用次数由 内容量 而非像素决定
内部分辨率上限	约 1500 px，更大的图被内部降采样	输入须切到 1500 px 内，否则糊字（Table）
并发能力	16 路并发无限流	时间非瓶颈，可细粒度切分换精度

2. 系统总览

两类图的物理形态与失败模式完全不同，按官方发布目录分流到两条独立流水线（不做长宽比自动分类，以免平白引入一类错误）：LONG 面条图宽度锁定约 1500 px、恰在模型有效分辨率内，可整宽送入、仅需纵向切分；TABLE 大表图宽达两万余像素、必须二维切分。TABLE 的 GT 内容量中位约为 LONG 的数倍，是计算量与难度的双重主体。两条流水线各自产出 Markdown/HTML，合并后按 file_name 排序写出 submission.csv。



图级并行用进程池（绕开 GIL，几何计算 CPU 密集），图内 API 调用用线程池并发，单图异常不影响整批。

3. TABLE 流水线

TABLE 路整体分三个阶段：Stage I 用纯几何手段把整图分解为若干**单一子表**；Stage II 在每个子表内估计行列骨架、按网格线切分并识别，同时对每个切块做一致性校验与修复；Stage III 把校验通过的切块按全局坐标重组为完整表格。

3.1 Stage I: 几何裁剪与子表切分

这一阶段完全不调用模型，输出是按阅读顺序排序的块序列 $[(k_i, b_i)]$ ， b_i 为包围盒， $k_i \in \{\text{text}, \text{title}, \text{table}\}$ 分别走纯文本识别、待裁定、网格识别。分三层：

第一层，剥离表外文字。页眉、页脚、水印、页码混入表格区域会污染网格估计。判据是三个条件的合取：矮（块高低于阈值）、窄（块宽显著小于主体）、且非最大连通分量。“窄”这一条尤其关键，它保护了三角形稀疏表底部那些又宽又矮的有效数据段不被误剥。

第二层，切分子表。子表有左右并排与上下堆叠两种排布，这是两个正交的维度，判据不同：先按竖直中缝切成若干竖栏，再在每栏内独立求上下切点。

- **左右并排：横线断裂。**竖线的默认含义是列分隔而非分表信号，以“竖线数量多”判定并排会使任何多列表格被误切；真正的信号是横线在某个 x 处断开。把所有横框线沿 x 轴的覆盖区间叠加成投影：单张表的横线横跨全宽、投影连成一片；投影中间出现明显缺口，说明每条横线只延伸到单栏宽就断了，即多个带框窄表并排印刷，缺口 midpoint 即中缝。无横框线的无框表一律判不并排。
- **上下堆叠：缝高 $\wedge$ 框线连续性。**缝高——先剔除纵向贯穿的竖线列（否则框线为每一行贡献约 10 px 墨迹，无一行能判为空白），再将整行几乎无墨迹的像素行判为空白行、连续空白行构成空白段；以空白段高度的中位数为行缝基准 h_0 ，真子表缝需满足 $h_{\text{seam}} \geq k h_0$ ， $k = 2$ （两个分布之间在 $[1.1, 2.5]$ 内几乎无样本， $k = 2$ 落在空隙中央）。框线连续性——稀疏表底部的空行墨量同样为零，但边框与列线仍连续穿过，故要求缝内贯穿竖线不超过 2 条（真子表缝处为 0）。
 - 后备判据：与横向的横线断裂对偶，纵向另有一条**竖线断裂**判据——框线竖线在某 y 处贯穿率极低即判为缝，切点取断裂带顶部使标题归下方子表，专救空白段被稀疏行打断、缝高不达标的有框密表。
 - 护栏：单栏切点数超过阈值即判定检测失效，放弃该栏全部切点、整栏送入识别（列错位表的行缝会被行号列分割出大量假缝）。漏切尚可由识别环节补救，过度切分则不可挽回。

第三层，块类型判定。第二层只切不判。两类错误的代价不对称：文字判成表格可恢复（读出的 `<td>` 寥寥无几，Stage II 自动降级为文本，代价一次调用），表格判为文字则整片数据退化为纯文本，不可恢复。故几何判据取保守下界，其余交模型裁定：

- **骨架初判。**按段内行列骨架的格数分流：低于 30 格恒判 `text`（实测垃圾段不超过 54 格、真表不低于 78 格）；另有一条后备规则把结构上明显不构成表格的矮条与窄条同样归入 `text`；其余判为表格段。
- **剥表顶标题。**表格段还需剥掉紧贴表体、第二层切不开的标题行：有横框线时最上一条即表格上边界、其上为标题（上方无文字行则表体贴顶、不剥）；无框表改看行墨迹的横向分布，数据行铺满全宽且分裂成多段墨迹、标题局部集中，从顶剥到第一个数据行。剥深超过 3 行时要求前缀各行都呈非数据行形态才执行，以免把三角表顶部的稀疏数据行当标题剥走。
- **模型复核类型**（在 Stage II 执行）。剥出的块标记为 `title` 单独送一次模型：返回 `<td>` 数不少于 6，说明它其实是被误剥的表头或数据行，改判为表格碎片交 Stage III 拼回表顶（见 3.4）；否则取其纯文本。

3.2 Stage II：行列估计与网格切分

行列骨架估计。切块、一致性判定与重组都以行列骨架为坐标基准。行、列两个方向共用前两步，各自再补一条本方向专用的规则：

- （两方向共用）投影前去除交叉方向的框线。估计行边界前先去纵纵向贯穿的竖线列，估计列边界前先去横横向贯穿的横线行，否则框线会使每一行（列）都带有墨迹，单元格之间的空白缝隙全部消失。紧贴图像边缘的空白区属页边距而非单元格间缝，同样不产生边界。
- （两方向共用）有框 / 无框二分。以墨柱法检测框线：某方向最长连续墨段超过阈值的像素行（列）判为框线，阈值随表高自适应，否则矮表的短框线会被漏检。若框线数量足够多且明显多于空白缝候选，判为有框表，**框线本身即单元格边界**；否则判为无框表，改用单元格之间的空白缝定界，缝中心为边界、首尾补入图像边界。
- （列方向）按列距均匀性选缝，并剔除首行。枚举候选缝宽的下限，取使列间距最集中于其中位数的取值：真实列缝产生近似等距的列，而大字号表中的宽字符间隙与标签文字的字间空白会破坏等距，固定阈值在这两类表上必然二选一失效。此外无条件剔除首行——首行多为跨列标题或说明，其字符间隙在主体无墨区伪装成列缝，使列数显著虚增，列结构应由主体数据行决定。
- （行方向）近邻边界消歧。表内行距均匀，两条相邻边界的间距不足行距中位数的一半时不可能同时为真，其中必有一条穿过文字。保留证据更强的一条（位于横线上者优先，否则空白段更宽者），迭代至不再存在近邻对。

骨架修复。估出的骨架在切分前再经两步修复。

- 补插漏检的列线。两列之间若没有框线、数值又排得密，空白缝检测找不到分界，两列被并成一个宽列。判据：逐行统计该宽列内墨迹的横向分布，若各行的墨迹都落在同样的两个 x 区带内、区带间的空隙跨行对齐，即判为两列，在空隙处插入列界。若多数行的两个区带同时有墨，则无法排除“一个数字被中间空隙分开”，此时裁出该列条带单独送模型识别一次，模型同样读出两列才插线。
- 删除多检的列线。装饰性双线之间的细缝会被误检为一条列线，凭空多出一个窄列（真值中并无此列）。判据：该列宽度不足中位列宽的一半，且列内墨迹几乎全部延伸到邻列——只有连通域完整落在条内的墨迹才计为本列自有，多检列线切出的窄条接到的都是邻列文字探入的笔画，而真正的窄列（如序号列）自有墨迹很多。命中后删除该窄条与文字所属一侧之间的边界，窄条并回该列。

若骨架仍严重错位（局部列带的行结构与全表明显不一致），则不在错误坐标系上强行重建，而是整体转入自由分块识别的回退路线，只在训练集上触发。

切块。在单个子表内，切分必须严格落在检测到的网格线上，使每个切块（tile）包含**整数行、整数列**。切块尺寸上限为 25 行 $\times$ 15 列，由单变量隔离实验确定。

空白块预判。墨量极低的切块不送模型，直接按已知网格填充空单元格。既省调用（100 张图约 1125 次），也根除大模型输出幻觉。

小字上采样。定义切块的等效格边长

$$e = \sqrt{\frac{A_{\text{eff}}}{n_r \cdot n_c}}$$

其中 A_{eff} 为去除页边距后的面积， n_r, n_c 为行列数。当 $e < 40$ px 时判为密集小字，放大倍数取

$$s = \text{clamp}\left(\frac{75}{e}, 1, 3\right)$$

即把格边长调整到目标值 75 px。使用二维格面积而非一维列间距，是为了正确识别“列稀行密”型表格。

3.3 一致性判定与确定性重读

TABLE 路的核心机制，也是最终版本相对前代的主要增益来源，分三段：**几何期望** $\rightarrow$ **一致性判定** $\rightarrow$ **确定性重读**。

几何期望。对每个切块，由墨迹几何独立计算出一个期望结构，不依赖模型输出。格 (i, j) 是否有内容按严、松两档二值化判定，两档均先扣除穿过格内的框线及其抗锯齿边，再看剩余墨迹：严档（灰度 < 128 ）捕正文浓墨，得 $\text{ink}(i, j)$ ；仅当严档判空时再以松档（ < 180 ）复判一次，得 $\text{gray}(i, j)$ 。格占位取二者之并

$$\text{cell}(i, j) = \text{ink}(i, j) \vee \text{gray}(i, j)$$

松档下淡灰框线与浅色底纹一并显影，扣线门限须相应提高，两档因而互不包含：单用严档会漏掉整表印成浅灰的数字（正文全部隐形、期望欠数），单用松档则会把框线灰边计为内容。期望结构 E 即全部 $\text{cell}(i, j)$ 构成的二值占位图：哪些行含有内容、每行的哪些格有墨。

一致性判定。模型返回的解析结果需满足**占位恒等**：非空行数等于 $|E|$ ，且逐格满足

$$\forall(i, j) : \quad \text{非空}(i, j) \iff \text{cell}(i, j)$$

判定零容差：判据必须逐格，仅约束列数上界或每行宽度都只是占位图的一维投影，会给“重复输出藏身其间”和“中段占位缺失、内容整体错位”留下窗口。仍捕获不到的是“占位完全相同、仅数值读错”，需内容层面的数值平滑性验证，属未来工作。

确定性重读。判定不通过的切块只触发一个修复动作：整块回退至本地 PP-OCrv6s 逐格识别——期望占位为真的

格按几何边界裁剪、放大后本地识别（，为假的格置空，再按骨架填入规整网格。行列位置完全由骨架给定，本地模型只回答“这个格子里是什么”，计数类错误（行数爆炸、漏行、跳读）在结构上不可能发生；而单调重复区正是 API 最易失稳、逐格识别最易处理的输入。这一个通用动作替换了前代针对各类病态编写的七个专用修复模块，代码减少约 370 行而覆盖更完整。

3.4 Stage III：二维重组

重组把各切块的识别结果按全局行列坐标合并为完整表格，并处理以下结构问题：

表头重建。以“列号行”为锚点：在前三行中寻找连续整数游程长度不小于 5 的行，其上一行按内容形态分三种情况处理——单个短簇折入下方角格形成斜线表头；全为短簇识别为双行表头（左侧标签上下合并为 `rowspan`，右侧标签簇发 `colspan`）；含长簇（超过 10 字）则整行提升为表格标题。列号行以上的所有行一律标题化。表头中横跨多列的文字另按内容簇合成 `colspan`：每个文字格向右合并连续空档，直至下一个文字格，行首空档并入首格。此处设**数据行守卫**——训练集中带尾部空单元格的行首全部是数字为主的数据行，GT 保留空 `<td>` 而非合成 `colspan`，故非空格中数字格占多数时不做整形。

编号序列补齐。行号列与列号行的数字往往印得很淡，本地重读时会漏识成空格，在等差序列中留下空洞。按局部连续段线性补齐（横向限前 3 行、纵向限前 4 列，仅补 0-9）。必须按局部段而非全局拟合：并排双面板的两段列号行截距不同，全局拟合会同时算错两段。

误剥表格行的回接。Stage I 把表格上方的文字当标题切走时，有时会连表头行一起切走；Stage II 的复核发现该块含多个单元格，判定它是表格行而非标题。此处把它接回下一张表，作为该表的首行。若其后没有表格可接，则作为普通文本输出，不允许自成一张表——表数多出一张会使全篇的表格配对错位。

4. LONG 流水线

LONG 路的结构问题远轻于 TABLE：面条图宽度锁定 1500 px，恰好落在模型的有效分辨率内，横向不需要切分。核心挑战是切点选择、接缝处理与标题层级。

4.1 自适应切分

不按固定高度均匀切分，而是求每行墨量的水平投影，使切点落在**行间空白处**、即两文字行之间。这直接消除了接缝处的半行残字，后者是阅读流与文本编辑距离的主要失分源。切分只沿**高度方向**进行，目标条高 5000 px。

表格避让。条带内若检测到表格，切点应绕开，避免把一张表切成两半。表格检测按“线”而非按像素列计数（一条框线通常占 2 个相邻像素列，按列计数会高估一倍），且逐带独立核验，要求贯穿竖线不少于 3 条（左边框 + 列线 + 右边框），阈值取 2 会把双栏排版的分栏线误判为表格。

附带一条约束：**带高超过目标条高的表不避让**。该表本就超出条带高度，避让换不到完整性，反而顶高切点、造出异常短条带并引发连锁位移。

4.2 接缝拼接

拼接需处理两类情形：

普通接缝。相邻条带以不重叠的坐标裁出，正文接缝只需续行。

跨条表格。长表格中部没有行间空白，切点必然落在表内，上一条被闭合为 `</table>`、下一条重新开启 `<table>`，需跨条重连。另一类是切点穿过某个长单元格，该格前半部分被读到表外成为孤儿文本；识别条件为上条已闭表、孤儿文本很短且以连接标点结尾、两侧表格列数相同，满足则把孤儿前缀塞回下条首行的末单元格。

4.3 标题层级：文本定级与几何校准

模型按条带识别，# 的层级是各条带内的局部判断，拼成整篇后同一编号序列常在条带边界被重置。层级处理分两步：先由文本形态给出条带内的相对层级，再在整篇上结合原图排版证据校准，以后者为准。

文本定级 (heading_norm) 采用栈模型：维护一个大纲栈，按编号格式的连续性判断当前标题是同级、下钻还是回到上级。关键修正是，当遇到一个此前出现过的编号序列时，采用该序列的**历史层级**，而非相邻标题层级加减一。后者会在“深层游走后回到浅层”的情形下持续漂移。

此外还有三条针对性处理：约 56% 的文档中，封面大标题被模型拆成若干无 # 的普通段落，需合并并提升为一级标题；目录条日常被输出为 * 1.1 ... 形式的列表项，需在目录区转为标题；表内满宽横幅行的 colspan 需归一化（训练集 GT 中该写法有 102 例，写作裸 <td> 的仅 5 例）。

几何校准 (geom_heading) 是 LONG 路的最后一步：在整篇拼合完成后重定级全部标题，并以其结果为准。主逻辑仍是编号格式关系——栈跟踪祖先深度，新格式为父级 +1、同格式为兄弟同级、x.y 按点数取绝对深度，缺父级时按中文编号约定的深浅兜底。**图像证据只用在两处，且都只裁定一级标题：**

- 淡横线：标题上下的浅色分隔线是“章”的排版标记，横线上方的标题强制提升为 H1（可救回模型整章漏标），其下内容顺延至 H2；带 x.y 编号的标题深度已由点数定死，不受横线覆盖。
- 字号：取行内 core 高度（墨量不低于中位数 0.6 倍的像素行数），相对本文正文中位数归一，每个文档独立计算。无编号的中段标题若字号不低于正文的 1.25 倍，判为真篇名、保留 H1；否则判为模型把强调句读成了标题，删除该标题标记。

逐行识别使用同一个本地小模型，结果按图片名缓存。这一步必须放在拼接与文本定级之后：整篇拼合完成，字号与编号序列才具有全局可比性。

5. 参数设置

参数	值	依据
MAX_CONCURRENCY	32	图内 API 调用的线程并发数
POOL_PROCS	6	图级并行进程数（几何计算 CPU 密集，进程池绕开 GIL）
API_TIMEOUT	40 s	单次请求超时；服务端过载时“收下不回”，久等无收益
API_RETRIES	3	失败重试次数，失败项由缓存重跑补齐
DEGEN_RETRIES	1	复读退化的额外重试；退化是确定性的，多试无收益
LONG target_h	5000 px	延迟随高度线性增长，此处为延迟与条数的平衡点
LONG MIN_RULES	3 条	表格至少含左框、列线、右框；取 2 会把双栏分栏线误判成表
TABLE 切块上限	25 行 × 15 列	单变量隔离实验确定
上采样阈值 / 上限	$e < 40 \text{ px} / 3\times$	目标格边长 75 px；二维格面积可识别“列稀行密”型表格
并排中缝：横线投影缺口	$> 0.03W$	横线行判据为最长墨段 $\geq 0.25W$ 、计入投影的长段为 $\geq 0.1W$
单栏切点数上限	15	正常图不超过 13，检测失效时跃至 54，阈值取在空隙中
子表缝高倍数 k	2	真缝为行缝的 2.5–10 倍、空行缝约 1 倍， $k = 2$ 落在两分布的空隙中央
缝内贯穿竖线上限	2 条	真子表缝处框线断开，表内空行处框线仍连续贯穿
BIN_INK / BIN_FAINT / BIN_LINE	128 / 180 / 170	严档 (128) 捕正文浓墨、松档 (180) 含淡灰框线与浅标题；170 专供并排面板中缝与横线检测

全部阈值均为几何或统计维度上的通用判据，不含针对特定图像的分支或白名单。

6. 迭代历程

TABLE 路经历了三次架构级迭代，LONG 路的演进相对平缓。线上分数一律按分半提交口径给出：两路分别提交、另一半置空，因而可分别归因。本地分数则是在对应数据集（100 张训练表 / 100 张训练面条图）上按官方三项等权公式算得的均值（满分 100），与线上分数口径不同，两者不可比。

6.1 TABLE 第一代：整表切分（线上 TABLE 约 40）

第一代把每张图当作单个表格处理：检测网格线，按网格切分为切块，逐块识别后按坐标拼回。

这一代的起点是一个决定性的实验结论。最初的行带切分（只切高度、保留全宽）在中位表上切了 16 个带，TEDS 仅 0.49，内容只读回 62k/74k 字符，原因正是宽度超过模型内部分辨率上限而被降采样。改为原生分辨率的二维方块切分后，内容召回率的对比如下：

切块尺寸	中位表切块数	内容召回率
1100 px	35	0.979
1500 px	20	0.998
2000 px	12	0.920
2400 px	6	0.808

1500 px 锁定为切块上限。至此问题从“内容读不回来”转化为“读回来的碎块如何拼成正确结构”。

在这一架构上我们做了十余轮改进：多子表分段（利用模型自身返回的多个 <table>）、列数封顶、超大切块强制细分、截断表恢复（补齐未闭合标签而非整块丢弃）、展平幻觉过滤、空白判据阈值调整、全局列宽一致、尾部全空列裁剪等。这一代结束时线上 TABLE 半边约 40（同期 LONG 约 40.5，按分半口径反推），其中密集小字上采样是单项最大增益。

瓶颈与放弃的原因。到这一代后期，改进变得越来越困难且互相冲突，修好一类表往往打坏另一类。根本原因是这一代**没有可靠的行列估计**：所有结构决策都依赖模型返回的行列数，而模型在密表上的行列分割本身就不稳定（同一横带的各列块可能读出 11、12、0 等互不相同的行数）。在一个不可靠的基准上做修补，只能不断增加特例分支，越改越乱。期间也验证过用检测到的网格骨架**强制**约束模型行数，但当时检测精度不足（83 列对真值 70），硬约束反而把原本正确的切块也搅乱（均值 0.38 → 0.09），由此确立“检测结果只作软先验、不作硬约束”的原则。

6.2 TABLE 第二代：全面几何化（线上 TABLE 52）

第二代是架构重写，核心是把结构决策从模型转移到几何：

- 1. **剥离全部表外文字**：页眉、页脚、水印、页码，以及子表标题，一律先切出来单独走纯文本识别；
- 2. **子表切分**：用缝高与框线连续性两条判据把整图切成若干单一子表，每次只处理一个（此前试过固定间隙、覆盖度、自适应、降采样问模型、列指纹五种检测均不稳健，根因是三角表的自然收窄与真子表边界在投影上不可区分，过切会破坏当时已达 0.7-0.95 的 74 张单表）；
- 3. **行列估计**：在单个子表内独立完成行列网格的几何估计，作为后续所有重组步骤的坐标基准。

第 2 项的效果最为显著：被切分的多子表子集平均 TEDS 从 69.66 提升至 83.94（+14.28），而未被切分的单表输出逐字不变，这是一个零风险的改动，因为判据不满足时流程完全退化为原路径。

行列估计的精度在这一代达到可用水平：有框表的列估计 51/52 精确，无框表的行估计精确率从 77/113 提升至 106/113、列估计从 23/96 提升至 84/96 且零多估。

残留问题。即使行列估计正确，模型内容仍可能失配，形态多样：幻读、展平、少行、少列。第二代针对每种形态分别编写修复逻辑，对半重读、截断补救、展平三路修复、列收缩拆分等共七个模块。这些模块彼此耦合、触发条件互相干扰，且各自只覆盖已知形态，遇新形态即失效。

6.3 TABLE 第三代：统一的一致性判定与确定性重读（线上 TABLE 54.24）

第三代的洞察是：上述七种失配形态是**同一个问题的不同表现**，模型返回的结构与几何期望不一致。既然如此，就不需要分别识别每种形态，只需要一个判据判断“是否一致”，和一个动作处理“不一致”。

于是有了 3.3 节的三段式：几何期望给出零容差的占位恒等判定，不通过则整块回退至本地小模型逐格重读。七个专用修复模块全部删除，代码减少约 370 行。

线上 TABLE 半边达到 **54.2354**。在 B 榜集合上，223 个切块触发了本地重读，残余未解决的审计报告警仅 4 条。

6.4 LONG 路的演进（线上 LONG 40.49 → 42.04）

LONG 路的基础流水线（行间空白切分 + 接缝拼接）在早期就达到了较高水平且长期无回归。后续改动集中在两处，合计使线上 LONG 半边从 40.49 提升至 **42.0396**。

标题处理。依次实现了栈模型文本定级、封面标题合并提升、目录列表项转标题、几何校准。

长文内表格。面条图中散布着表格，而表格在评分中权重较高。这方面的改动包括表格避让切点、跨条表格重连、孤儿单元格回收、满宽横幅行的 colspan 归一。

7. 评测与可审核性

评测结果。下表为训练集（带 GT）的几何评测与线上分数。

维度	结果
子表切分成功率（最终表数 = GT）	97/97 = 100%（含多子表 43/43），漏切率 0
行列估计准确率	无框表行 106/113、列 84/96（零多估），有框表列 51/52
本地回退触发（B 榜）	223 个切块回退至格级重读，残余审计报告警仅 4 条
最终线上分数	TABLE 54.2354 、LONG 42.0396

切分成功率统计的是“表数正确”（几何有意多切、Stage III 全部收敛回 GT，如 GT=4 经几何 8 段合回 4），而非逐格内容；它与行列估计精度为训练集数字，B 榜无 GT 不可计。

可视化审核。到高分段，本地指标（TEDS、编辑距离）趋于饱和：约 3.5 分的差距在任何本地指标上都不可见，自动分数已不足以驱动进一步优化。为此构建可视化审核，把人眼核对变成可操作的优化信号：行列骨架审核工具（eval_grid）把检测到的行边界（绿线）、列边界（红线）、tile 切分带（蓝线）叠在原图上，并对照 GT 的 tr/td 数给出准确度，直观暴露错切与漏切；重构对照工具（render_pred_grids）把预测的 HTML 表格渲染成图，与原图并列逐表核对结构还原。二者暴露了纯数字掩盖的错误（如占位相同、仅数值读错），指导了后续的定向修复。

8. 推理资源与时间

不依赖 GPU。全流程为 CPU 计算、远端 API 调用，以及一个 5.27M 参数的本地 CPU 小模型。

项	配置
硬件	24 核 CPU / 62 GB 内存 / 无 GPU；建议不低于 8 核、16 GB
软件	Python 3.11 / Linux；依赖见 requirements.txt，无需编译扩展
磁盘	不低于 2 GB（API 结果与本地识别结果的落盘缓存）
并发	图级 6 进程 × 图内 32 线程

推理时间：B 榜实测两个半边各约 10 分钟（各 50 张，并发 32）。引入本地小模型承担残差重读与几何校准后两侧均略有增加，几何校准冷启动约 5 秒/张，残差重读仅在判定不通过的切块上触发。全量 100 张按 30 分钟预估，留足余量亦在 1 小时以内，远低于 3 小时上限。

耗时的主要变数在 API 侧负载而非本地计算，这也是 API_TIMEOUT 设为 40 秒快速放弃、依靠缓存重跑补齐失败项的原因。

未使用 ensemble。全流程只有 FinixDoc-VL 一个视觉语言模型，无多模型投票、无多次采样融合。本地小模型

不参与投票，仅在一致性判定不通过时作为确定性执行体接管，二者是串行的职责划分而非并行的结果融合。全部增益来自几何前处理与结构后处理。

9. 主要创新点

表格感知的自适应切分。面条图在行间空白处切分、密表严格按几何网格线切出整数行列的二维块，切点永不落在文字行或单元格内部。好处是从根本上消除接缝处的半行残字与密表相邻块的行数抖动，这两类正是按固定高度均匀切分的主要失分源。

原生分辨率的自适应上采样。按切块的等效格边长估计文字密度，对密集小字块放大到目标格边长后再送识别。好处是双赢：既避免超过模型有效分辨率的输入被内部降采样糊掉小字，又反直觉地缩短了模型解码延迟，密集小字块在原图分辨率下既慢又错、上采样后既快又准，无需在精度与延迟间取舍。

几何骨架的自适应行列估计。由墨迹几何独立估出行列骨架（含补插漏检列线、删除多检列线），不依赖模型返回的行列数。好处是把结构决策权从模型收回代码：模型退化为无状态的单格问答，行数爆炸、漏行、整块展平这类计数错误的发生条件本身不复存在，这是与“增加容错、冗余投票”等常规纠错的根本区别。

用几何占位判断 API 对错的零容差校验。从原图墨迹算出每格“应有内容 / 应空”的期望，要求模型输出逐格恒等匹配，否则整块判不合格。好处是不依赖模型置信度、以原图物理证据当质量门，能统一发现幻读、漏行、错位、展平；且判据越严格，所需的修复逻辑越少——七个专用修复模块由一个通用动作取代。

本地小模型的确定性回退。不合格块整块回退至本地 PP-OCrv6s 逐格识别，行列位置由骨架给定、模型只回答单格内容。好处是单调重复区（API 最易失稳的输入）恰是逐格识别最稳的场景，输出确定、可复现；且只在少数不合格块触发，CPU 成本可控。

空白块预判绕开编造幻觉。因接口无法用提示词告知“此处为空”，改用墨量投影自行预判空白块、直接按骨架填空而不送模型。好处是既省下大量调用，又根除了模型对空单元格的编造，一个由接口约束反向催生、比“能用提示词”更鲁棒的前处理。

10. 局限与展望

局限。本方案的几何判据主要面向印刷体、规则或半规则的金融表格，面对更复杂多样的表格仍有局限：无框表的行列骨架仅靠墨迹推断，遇到列距不均、稀疏交错的花式时精度会下降；跨页延续的表格、图文混排、旋转或透视变形的截图、手写与彩色底纹表格尚未专门处理；几何骨架一旦估计错误，误差会传递到后续的局部识别。残余失分则主要在识别层面，以形近字与断行位置差异为主，属内容细节而非结构问题。

更根本的一条是**方案本身偏复杂**：整套流程由多层几何判据串联而成，规则数量多、层与层之间有依赖，前一层的切分错误会沿链条放大。这些判据虽然都建立在统计分布而非个案之上，但迁移到新版式时需要逐层重新核验，维护成本高于端到端模型。第三代用一个通用的一致性判定替换七个专用修复模块，正是朝简化方向的一次收敛，这一方向仍有继续压缩的空间。

展望。可引入参数预算内的轻量版面模型提升复杂版式的鲁棒性，为切块注入表头与标签列锚点，以数值平滑性等结构先验做离群校验。更一般地，本方案“几何定位 + 模型识别 + 零容差校验 + 确定性回退”的模式不限于文档解析：任何“大模型在长序列上丢失位置追踪”的任务（长表格生成、长列表处理、多轮结构化抽取）都可沿用，把位置控制权交还代码、模型只做稳定的局部判断。

11. 合规声明

- **大模型使用：**全工程唯一的视觉语言模型接口是主办方提供的 FinixDoc-VL API，调用点集中于 `src/common/api_client.py`（两处 `requests.post`），无任何其它第三方模型 API、SDK 或网络地址。
- **提示词：**该 API 请求体仅含 `userId / apiKey / fileName` + 图片文件，不接受文本提示词参数，因此本方案不存在提示词配置文件。
- **本地小模型：**PP-OCrv6_rec_small.onnx, 5.27M 参数（低于 10M 上限），CPU + onnxruntime 推理。同一引擎初始化时载入的检测与方向分类模型在调用时均置 `use_det=False / use_cls=False`，不参与推理；三者参数量合计亦仅 7.72M。模型文件由 rapidocr 提供，安装阶段联网下载，不需要手动准备权重。

- 无结果硬编码：代码中不含针对特定测试图像的固定输出、白名单或标识符分支。
- 一键复现：`bash run.sh` 单条命令产出完整的 `submission.csv`。